

PRD — Model Router & Fallback Berantai (BE-6 + BE-4)

Versi: 1.0 · **Tanggal:** 12 Juli 2026 · **Status:** Draft · **Target rilis:** v0.2 · **Induk:** PRD-Voice-Assistant (global)

1. Ringkasan

Fitur ini membuat backend mampu memakai **beberapa model LLM sekaligus secara cerdas**, lewat dua mekanisme yang bekerja berlapis:

1. **Router per tugas (BE-6)** — setiap jenis pekerjaan otomatis diarahkan ke model yang paling cocok: voice → Groq/Llama (tercepat), chat → Gemini Flash (terpintar), tugas latar → model kecil termurah.
2. **Fallback berantai (BE-4)** — jika model pilihan gagal (kuota habis, error, timeout), permintaan otomatis dialihkan ke model cadangan berikutnya **tanpa user menyadarinya**.

Seluruhnya dibangun di atas **LiteLLM** sebagai lapisan abstraksi tunggal, dengan konfigurasi terpusat di satu file YAML.

2. Masalah yang Diselesaikan

1. **Latensi voice terlalu tinggi** bila memakai satu model untuk semua — mode suara butuh first-token secepat mungkin agar percakapan terasa hidup; Gemini pintar tapi Groq jauh lebih cepat untuk ini.
2. **Kuota free tier terbatas dan bisa habis kapan saja** — tanpa fallback, asisten mati total begitu limit tercapai; dengan fallback, layanan tetap hidup.
3. **Boros kuota model utama untuk tugas remeh** — pekerjaan latar (merangkum, klasifikasi pendek) tidak butuh model pintar; router mengalihkannya ke model murah sehingga kuota Gemini/Groq awet untuk percakapan.

3. Goals & Non-Goals

Goals:

1. Setiap request LLM diarahkan otomatis ke model sesuai jenis tugasnya, tanpa kode pemanggil tahu-menahu soal provider.

2. Kegagalan provider ter-recover otomatis < 2 detik (pindah ke cadangan berikutnya).
3. Ganti pemetaan tugas→model cukup edit satu file config, tanpa ubah kode.
4. User bisa override router secara manual via `/model`, dan melihat kondisi semua provider via `/status`.
5. Pemakaian kuota tiap provider terpantau agar fallback bisa **proaktif** (pindah sebelum limit), bukan hanya reaktif.

Non-Goals: paralel/ensemble (kirim satu prompt ke banyak model sekaligus) tidak dibangun — paling banter jadi eksperimen `/compare` di masa depan; load balancing antar API key; caching respons; streaming antar-provider mid-response (sekali respons dimulai di satu model, dia selesaikan — fallback hanya terjadi sebelum/di awal respons).

4. Konsep Inti

4.1 Task Type — bukan model, tapi jenis pekerjaan

Semua kode di backend **tidak pernah menyebut nama model**. Yang disebut hanya *jenis tugas*:

Task type	Dipakai oleh	Kebutuhan utama	Model default
<code>voice</code>	Mode percakapan suara	Kecepatan (first-token < 300ms)	Groq · Llama 3.3 70B
<code>chat</code>	Mode chat teks	Kualitas + context panjang	Gemini 2.5 Flash
<code>background</code>	Tugas latar: rangkuman, klasifikasi, judul sesi	Murah	Gemini Flash-Lite

Contoh pemanggilan di kode (satu-satunya cara service lain memanggil LLM):

```
response = await llm.complete(task="voice", messages=[...], stream=True)
```

Router yang menerjemahkan `task="voice"` menjadi model konkret. Menambah task type baru di masa depan (misal `vision`) = tambah satu blok di config.

4.2 Rantai Fallback — tiap task punya antrean cadangan

Setiap task type memiliki **daftar model terurut**. Router selalu mencoba urutan teratas yang sehat:

voice: groq/llama-3.3-70b → gemini-2.5-flash → openrouter/llama-3.3:free

chat: gemini-2.5-flash → groq/llama-3.3-70b → openrouter/gemini-flash:free

background: gemini-flash-lite → openrouter/(model :free apa pun) → groq/llama-3.1-8b

Perhatikan rantainya **beda per task** — cadangan voice tetap mengutamakan yang cepat, cadangan chat mengutamakan yang pintar.

4.3 File Konfigurasi (sumber kebenaran tunggal)

`config/models.yaml`:

providers:

gemini:

api_key_env: GEMINI_API_KEY

rpm_limit: 10 # requests/menit free tier — dipantau lokal

rpd_limit: 250 # requests/hari

groq:

api_key_env: GROQ_API_KEY

rpm_limit: 30

rpd_limit: 1000

openrouter:

api_key_env: OPENROUTER_API_KEY

rpm_limit: 20

tasks:

voice:

chain:

- groq/llama-3.3-70b-versatile
- gemini/gemini-2.5-flash
- openrouter/meta-llama/llama-3.3-70b-instruct:free

timeout_first_token_ms: 1500

chat:

chain:

- gemini/gemini-2.5-flash
- groq/llama-3.3-70b-versatile
- openrouter/google/gemini-2.0-flash-exp:free

timeout_first_token_ms: 4000

background:

chain:

- gemini/gemini-flash-lite
- groq/llama-3.1-8b-instant

timeout_first_token_ms: 10000

fallback:

cooldown_seconds: 300 # provider gagal "diistirahatkan" 5 menit

proactive_threshold: 0.9 # pindah proaktif saat pemakaian kuota $\geq$ 90%

max_attempts_per_request: 3 # maksimal coba 3 model per request

Angka limit di atas adalah placeholder — **wajib diverifikasi dengan free tier terkini tiap provider saat implementasi**, karena kebijakan kuota mereka sering berubah.

5. Perilaku Detail

5.1 Alur satu request

llm.complete(task="chat", ...)

|



[Router] baca chain "chat" dari config

|



[Health check lokal] lewati model yang sedang cooldown

atau pemakaian kuotanya $\geq 90\%$ (proaktif)

|



[Coba model #1] — sukses —► stream respons ke pemanggil, catat pemakaian

| gagal (lihat 5.2)



[Tandai cooldown model #1] → [Coba model #2] → ... (maks 3 percobaan)

| semua gagal



[Error terkontrol] → pesan ramah ke user (lihat 5.4)

5.2 Apa yang dihitung "gagal" (memicu fallback)

Kondisi	Deteksi	Aksi
Kuota habis	HTTP 429 / pesan quota dari provider	Cooldown provider (bukan cuma model) 5 menit, lanjut ke cadangan
Server provider error	HTTP 5xx	Cooldown model 5 menit, lanjut
Timeout first-token	Melebihi <code>timeout_first_token_ms</code> task tsb.	Batalkan, lanjut ke cadangan
Koneksi gagal	Network error	Retry 1× cepat, lalu lanjut ke cadangan
Kuota hampir habis (proaktif)	Penghitung lokal $\geq 90\%$ dari rpm/rpd	Model dilewati sebelum dicoba

Yang **tidak** memicu fallback: error dari sisi kita (prompt invalid, 400) — itu bug, harus dilempar sebagai error biasa agar ketahuan; dan content-filter refusal — pindah model tidak menyelesaikan masalahnya.

5.3 Pemantauan kuota lokal

Backend menghitung sendiri pemakaian per provider (requests/menit dan requests/hari, disimpan di SQLite tabel `usage`), karena provider tidak selalu memberi tahu sisa kuota. Penghitung harian di-reset mengikuti zona waktu reset masing-masing provider. Data ini dipakai untuk fallback proaktif dan ditampilkan di `/status`.

5.4 Saat SEMUA cadangan gagal

Jangan pernah diam atau crash. Respons ke user: pesan ramah + jujur — *"Waduh, semua model AI-ku lagi kena limit / gangguan. Coba lagi beberapa menit ya 🙏"* — plus di mode voice, pesan ini diucapkan (pakai audio pre-cached, bukan TTS live yang justru butuh model). Event WS: `{"type": "error", "code": "all_providers_down", "retry_after_s": 300}` agar UI bisa menampilkan countdown.

5.5 Interaksi dengan mode voice (kasus paling sensitif)

Fallback di tengah percakapan suara harus nyaris tak terasa: timeout first-token voice dibuat ketat (1,5 dtk) supaya kegagalan terdeteksi cepat; saat pindah model, konteks percakapan

dikirim ulang penuh ke model baru sehingga obrolan tetap nyambung; dan **gaya bicara bisa sedikit berubah antar model** — ini diterima sebagai **trade-off** dan dicatat di `/status` model mana yang sedang aktif.

5.6 Override manual: `/model`

Perintah	Efek
<code>/model</code>	Tampilkan model aktif per task + daftar yang tersedia
<code>/model chat gemini</code>	Kunci task <code>chat</code> ke Gemini (pin) untuk sesi ini
<code>/model voice groq</code>	Kunci task <code>voice</code> ke Groq
<code>/model auto</code>	Kembali ke router otomatis

Aturan pin: berlaku per sesi (hilang saat `/clear`); jika model yang di-pin gagal, fallback **tetap jalan** (keandalan menang atas preferensi) tapi user diberi tahu: *"Gemini lagi limit, sementara kupindah ke Groq ya."*

5.7 `/status`

Menampilkan: model aktif per task (dan apakah sedang di-pin), kondisi tiap provider (● sehat / ● kuota $\geq 90\%$ / ● cooldown + sisa waktu), pemakaian hari ini per provider (request & estimasi token), dan jumlah kejadian fallback hari ini.

6. Desain Teknis

6.1 Struktur modul

backend/

└─ llm/

└─ `__init__.py` # expose: `llm.complete(task, messages, stream)`

└─ `router.py` # pemilihan model: chain + health + pin

└─ `providers.py` # wrapper LiteLLM, normalisasi error antar provider

└─ `health.py` # cooldown registry + penghitung kuota (SQLite)

└─ config.py # loader + validasi models.yaml (fail-fast saat boot)

Prinsip: **hanya paket `llm/` yang tahu soal provider**. Chat service, voice service, dan memory service semuanya cuma memanggil `llm.complete(task=...)`.

6.2 Normalisasi error

Tiap provider melempar error dengan format berbeda-beda. `providers.py` menerjemahkan semuanya ke satu taksonomi internal: `QuotaExceeded`, `ProviderDown`, `Timeout`, `BadRequest`, `ContentFiltered` — supaya logika fallback di `router.py` bersih dan tidak penuh if-else per provider.

6.3 Logging

Setiap request LLM dicatat satu baris: `task`, `model_dipilih`, `percobaan_ke`, `latensi_first_token`, `latensi_total`, `token_in/out`, `hasil` (`ok/fallback/failed`). Ini bahan utama untuk tuning chain dan debugging "kok tadi jawabannya beda gaya".

7. Rencana Kerja (estimasi total: 4–6 hari)

Hari	Pekerjaan
1	<code>config.py</code> + <code>models.yaml</code> + validasi fail-fast; wrapper LiteLLM dasar (<code>providers.py</code>) untuk 3 provider, verifikasi limit free tier terkini
2	<code>router.py</code> : resolusi chain per task + integrasi streaming; refactor chat & voice service agar memanggil <code>llm.complete(task=...)</code>
3	<code>health.py</code> : normalisasi error, cooldown, penghitung kuota SQLite, fallback reaktif
4	Fallback proaktif (threshold 90%), timeout first-token per task, pesan "semua down" + audio cached
5	<code>/model</code> (pin per sesi) + <code>/status</code> ; logging lengkap

Hari	Pekerjaan
6	Pengujian skenario (lihat §8) + tuning

8. Skenario Pengujian (acceptance)

1. **Routing dasar:** request **voice** terbukti dilayani Groq, **chat** oleh Gemini, **background** oleh Flash-Lite (cek log).
2. **Fallback reaktif:** matikan API key Gemini (bikin 401/429 buatan) → request chat otomatis dilayani Groq, user hanya melihat jawaban normal; log mencatat fallback.
3. **Cooldown:** setelah gagal, Gemini tidak dicoba lagi selama 5 menit; setelah itu dicoba kembali otomatis.
4. **Proaktif:** set **rpm_limit** kecil (misal 3), kirim 4 request cepat → request ke-4 langsung ke cadangan tanpa menyentuh Gemini.
5. **Semua down:** matikan semua key → user menerima pesan ramah + **retry_after**, app tidak crash, mode voice mengucapkan pesan error dari cache.
6. **Pin:** **/model chat groq** → semua chat ke Groq; Groq dimatikan → fallback tetap jalan + notifikasi ke user; **/model auto** mengembalikan router.
7. **Voice mid-conversation:** saat sesi voice berjalan, matikan provider aktif → percakapan berlanjut di model cadangan dengan konteks tetap nyambung, jeda < 3 detik.
8. **Config error:** **models.yaml** berisi model tak dikenal → backend menolak boot dengan pesan jelas (fail-fast), bukan error misterius saat runtime.

9. Metrik Keberhasilan

1. First-token voice (p90) < 500ms via Groq dalam kondisi normal.
2. Waktu pemulihan saat provider gagal < 2 detik (dari error terdeteksi sampai token pertama model cadangan).
3. Zero "asisten mati": selama 1 minggu pemakaian, tidak ada momen tanpa respons sama sekali (pesan "semua down" yang ramah tetap dihitung sebagai respons).
4. ≥ 80% tugas **background** dilayani model murah (bukan memakan kuota Gemini/Groq utama).

10. Risiko & Mitigasi

Risiko	Mitigasi
Limit free tier berubah sewaktu-waktu	Limit di config (bukan hard-code) + fallback reaktif tetap menangkap 429 walau angka config meleset

Risiko	Mitigasi
Gaya jawaban berubah saat fallback (membingungkan)	System prompt persona yang sama dikirim ke semua model; <code>/status</code> transparan soal model aktif
Penghitung kuota lokal tidak akurat (drift vs hitungan provider)	Threshold proaktif 90% memberi margin; 429 tetap ditangani reaktif sebagai jaring terakhir
Model <code>:free</code> OpenRouter datang-pergi	Chain OpenRouter diperlakukan best-effort (posisi terakhir); gagal resolve → dilewati tanpa drama
Konteks panjang tidak muat di model cadangan yang lebih kecil	Router memangkas riwayat (keep N terakhir + rangkuman) saat pindah ke model dengan context window lebih kecil

11. Ketergantungan

Butuh MVP v0.1 selesai (chat & voice service sudah jalan dengan satu model). Fitur ini adalah item pertama v0.2 dan menjadi fondasi bagi fitur v0.2 lain (memory summarizer memakai task `background`; pipeline Mode B memakai task `voice`).